Technical Product Management Assessment

Reddit – Technology, Data & Integration Analysis

- Technology Stack Analysis
- Database Design
- SQL Queries
- API Design

Prepared By:
Vaibhav Yadav

Executive Summary

Objective

This presentation evaluates Reddit's technology stack, data architecture, database design, SQL querying capabilities, and API integration approach — through the lens of a Product Manager working closely with engineering teams.

Areas Covered

Section	Objective
1.Reddit Technology Stack	Understand frontend, backend, database and infrastructure choices
2.Technology Trade-offs	Evaluate alternative technologies and engineering decisions
3.Database Schema Design	Design scalable data models for core Reddit features
4.SQL Problem Solving	Demonstrate data retrieval and analysis capabilities
5.Reddit × DuckDuckGo API Design	Propose a scalable partnership integration architecture

Key Evaluation Areas

- Technical understanding
- Scalability thinking
- Database design
- API design
- Engineering collaboration Mindset

PM × ENG Collaboration

Question - 1

Understanding Reddit as a Product

Know the building blocks before diving into the tech stack.

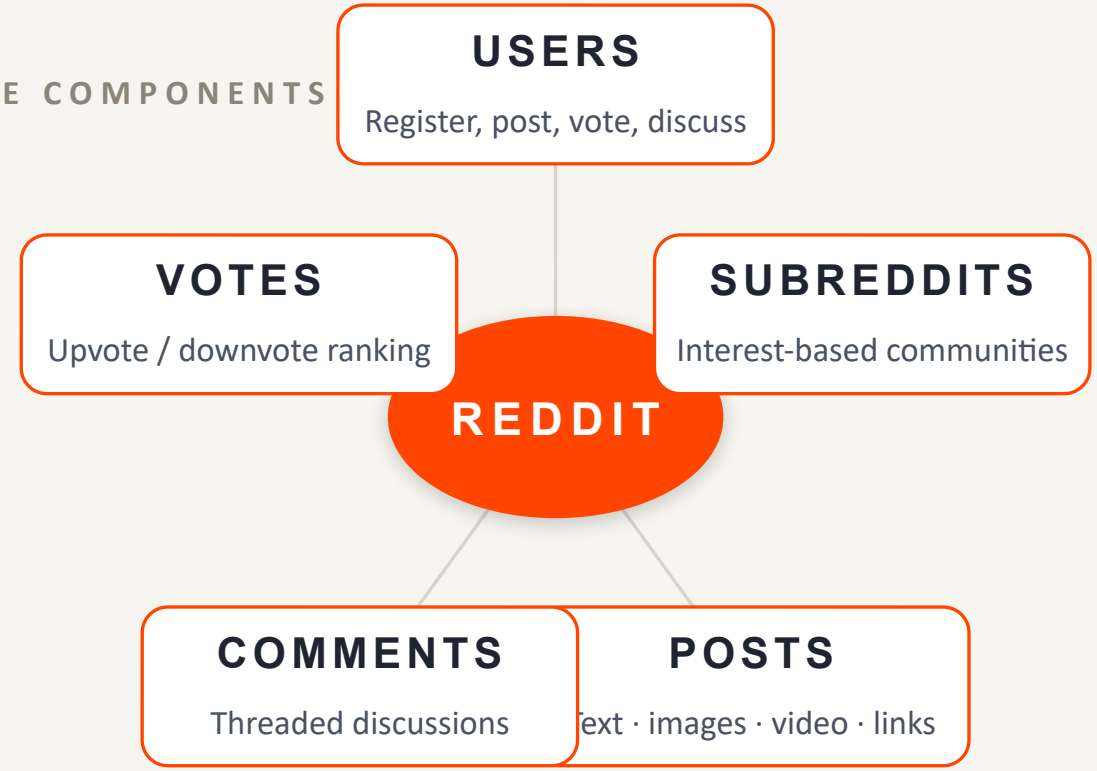
WHAT IS REDDIT?

A community-driven social platform where users discover, create, discuss, and vote on content inside topic-specific communities called subreddits.

The shift

*Traditional social networks orbit around **people and followers**. Reddit orbits around **communities and conversations**.*

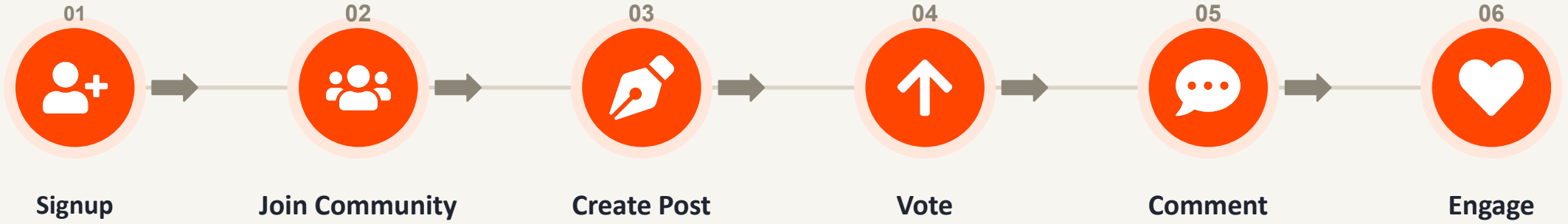
CORE COMPONENTS



ENGAGEMENT LOOP Users → Subreddits → Posts → Comments → Votes — together these five primitives are Reddit.

Reddit Core User Journey

Every tap, scroll, and click leaves a footprint the platform must capture.



ACTION → BUSINESS OUTCOME

Register	→ New user acquisition
Join subreddit	→ Community growth
Create post	→ Content generation
Upvote content	→ Content ranking
Comment	→ Engagement
Revisit subreddit	→ User retention

WHY THE JOURNEY MATTERS

Every interaction creates data that must be:

Captured | **Stored** | **Retrieved** | **Processed** | **Displayed**

These interactions directly shape Reddit's architecture and database design at scale.

KEY INSIGHT Every step in the journey is also a write, read, or rank operation — the user journey is the system load profile.

Feature-to-Technology Mapping

Where Product meets Engineering — every feature is a stack decision in disguise.

FEATURE → TECH CAPABILITY

FEATURE	TECHNOLOGY CAPABILITY REQUIRED
User Registration	→ Authentication Service + Database
Login	→ Authentication APIs
Create Subreddit	→ Backend Services + Database
Create Post	→ APIs + Database
Upvote / Downvote	→ Database + Cache
Add Comment	→ APIs + Database
Feed Loading	→ Cache + Database
Search Posts	→ Search Engine
Notifications	→ Event Processing Service
Content Ranking	→ Recommendation / Ranking Logic

EXAMPLE · USER CREATES A POST

USER SEES

Click "Create Post" → Post appears instantly

SYSTEM DOES

1. Frontend
2. API Request
3. Backend Service
4. Database Write
5. Response Returned
6. UI Updated

KEY INSIGHT One user click triggers six system steps — product simplicity is engineering complexity, well hidden.

How Reddit Delivers Content to Users

Architecture Flow



Request Lifecycle

1. User opens Reddit and requests content.
2. Frontend sends the request to Reddit's API layer.
3. API routes the request to the right backend service.
4. Hot data is served straight from the Redis cache.
5. On cache miss, backends query PostgreSQL.
6. Response is returned to the frontend and rendered.

KEY INSIGHT Reddit's layered architecture separates UI, logic, caching, and storage — enabling massive scale without coupling.

Reddit Technology Stack

The tools behind every layer — and why each one earns its spot.

LAYER	TECHNOLOGY	WHY USED
FRONTEND	**Lit (Web Components), Native iOS & Android**	Dynamic UI rendering for the user-facing feed.
FRONTEND	**TypeScript**	Static typing improves code reliability at scale.
BACKEND	**Python**	Core business logic and rapid service iteration.
BACKEND	**Go (Primary), Python (Legacy & AI/Data)**	High-performance services where latency matters.
API LAYER	**GraphQL**	Efficient, client-driven data fetching.
DATABASE	**PostgreSQL, Cassandra**	Reliable relational store for structured data.
CACHE	**Redis**	Microsecond reads for hot, frequently-accessed data.
CLOUD	**AWS**	Elastic infrastructure for global scale.
CDN	**Fastly**	Edge delivery for faster page loads worldwide.

BOTTOM INSIGHT Reddit blends relational storage, in-memory caching, and microservices to balance scalability, performance, and reliability.

Why These Technologies Make Sense for Reddit

Technology Choices and Business Rationale

Lit

FRONTEND

- Smaller than React
- Faster page loads
- Better performance on low-end devices
- Reduces browser-side JavaScript

Go

BACKEND

- Handles millions of concurrent users
- Faster execution than Python
- Better for microservices
- Lower infrastructure cost

PostgreSQL

DATABASE

- Strong relational data support
- Powers users, posts, comments, likes, communities
- Reliable transactional consistency

Redis

CACHE

- Reduces database load
- Speeds up feed generation
- Lower latency = better UX

AWS

CLOUD

- Horizontal scaling as traffic grows
- Global availability and reliability
- Less infrastructure to manage in-house

KEY INSIGHT Reddit's stack balances developer productivity, scalability, performance, and long-term maintainability.

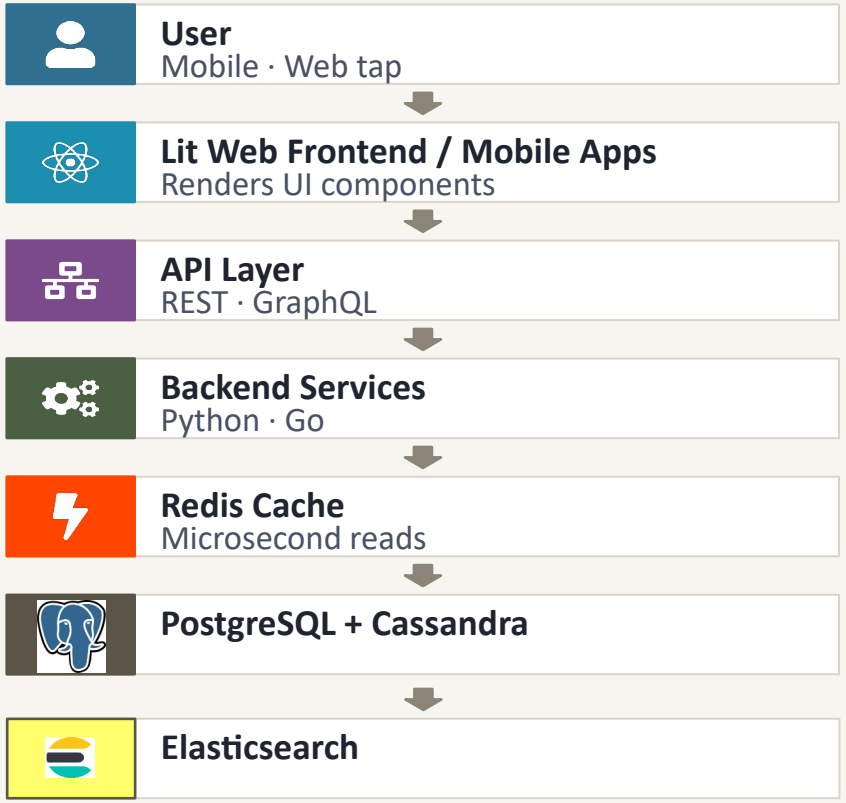
How Reddit Technologies Work Together

One request, six layers — choreographed in milliseconds.

EXAMPLE · VIEWING A REDDIT POST

1. User opens the Reddit app
2. React frontend sends a request
3. API gateway receives the request
4. Backend service (Python / Go) processes it
5. **Redis cache check**
   - HIT — Return data immediately
   - MISS — Go to database ↓
6. PostgreSQL database query (on cache miss)
7. Response returned to the frontend
8. User sees the post, comments and votes

TECHNOLOGY INTERACTION



KEY INSIGHT Cache-first design: most requests stop at Redis — PostgreSQL is the trusted fallback, not the default path.

Question - 2

Technology Alternatives & Evaluation

PostgreSQL vs MongoDB — could Reddit have picked the other one?

PostgreSQL
Relational · SQL

DATA STRUCTURE	**Tables & relationships**
JOINS	**Strong support**
CONSISTENCY	**High**
SCALABILITY	**Good**
FLEXIBILITY	**Medium**

VS

MongoDB
Document · NoSQL

DATA STRUCTURE	**JSON documents**
JOINS	**Limited**
CONSISTENCY	**Moderate**
SCALABILITY	**Excellent**
FLEXIBILITY	**High**

REDDIT USE CASE

Reddit's entities are deeply related:

USER → POST → COMMENT → LIKE

Joins are unavoidable.

RECOMMENDATION

PostgreSQL

Reddit relies on relational data and transactional consistency — PostgreSQL fits that profile natively.

PM INSIGHT

MongoDB would give more flexibility, but every cross-entity read (users + posts + comments + votes) becomes harder, not easier.

VERDICT Pick the database that matches the data shape. For Reddit's relational graph, PostgreSQL wins on joins and consistency — the trade-offs that actually matter.

Technology Trade-Off Analysis

Technology Comparison 2 · **Redis vs Memcached**

Head-to-Head Comparison

FACTOR	REDIS	MEMCACHED
Data Structures	Strings, Lists, Sets, Hashes	Key-Value only
Persistence	Supported	Not supported
Performance	Very fast	Extremely fast
Flexibility	High	Low
Scalability	High	Moderate

Reddit Use Case — what gets cached

- Popular posts
- Trending content
- Vote counts
- User sessions
- Community rankings

These workloads need richer data structures —
Redis supports all of them natively.

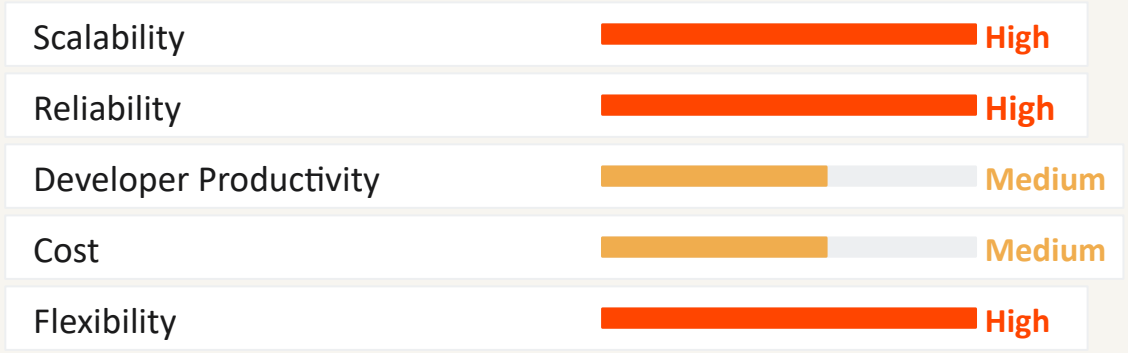
RECOMMENDATION

Choose Redis — one platform covers it all.

Caching · Sessions · Real-time counters · Leaderboards · Ranking systems

Redis = caching + state + analytics in a single technology footprint.

PM Decision Framework



KEY TAKEAWAY Don't pick tech by popularity — pick by product fit: requirements, scale, data shape, performance, future growth.

Question - 3

Database Requirements Analysis

Identify the core entities required to support Reddit's primary user interactions.

Features → Data Required

FEATURE	DATA REQUIRED
User Registration	User information
Create Subreddit	Community information
Create Post	Post information
Like Post	Vote information
Add Comment	Comment information

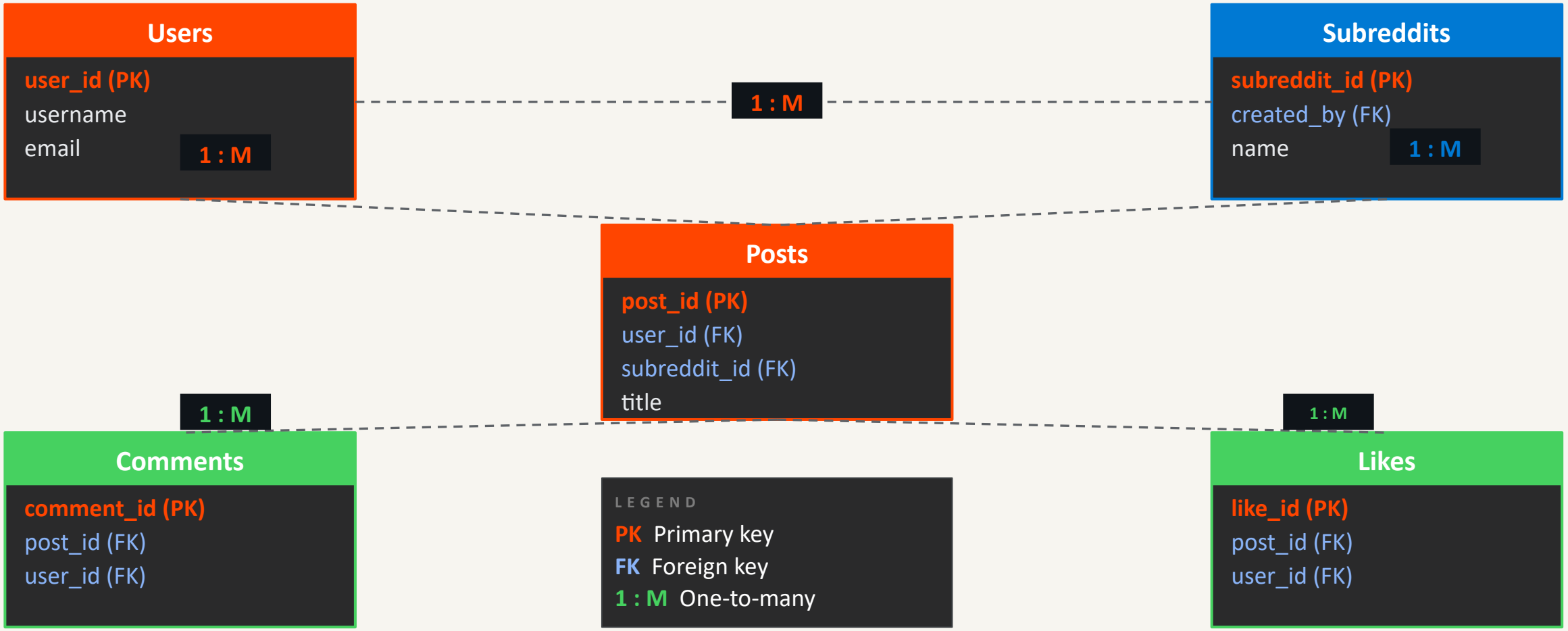
Core Entities Identified

1. **Users**
   Registered user profiles
2. **Subreddits**
   Community information
3. **Posts**
   Content created by users
4. **Comments**
   Discussion data
5. **Likes**
   User voting activity

KEY OBSERVATION Every Reddit feature maps to one of five entities — these form the foundation of the database architecture.

Entity Relationship Diagram (ERD)

How the five entities connect — primary keys, foreign keys, cardinality.



ERD INSIGHT Users sit at the top of every relationshp- they create subreddits, posts, comments, and likes.

Database Schema Design

Column-by-column definitions for the five core tables.

Users Table

COLUMN	TYPE
user_id	INT (PK)
username	VARCHAR
email	VARCHAR
password_hash	VARCHAR
created_at	TIMESTAMP

Subreddits Table

COLUMN	TYPE
subreddit_id	INT (PK)
subreddit_name	VARCHAR
description	TEXT
created_by	INT (FK)
created_at	TIMESTAMP

Posts Table

COLUMN	TYPE
post_id	INT (PK)
subreddit_id	INT (FK)
user_id	INT (FK)
title	VARCHAR
description	TEXT
created_at	TIMESTAMP

Comments Table

COLUMN	TYPE
comment_id	INT (PK)
post_id	INT (FK)
user_id	INT (FK)
comment_text	TEXT
created_at	TIMESTAMP

Likes Table

COLUMN	TYPE
like_id	INT (PK)
post_id	INT (FK)
user_id	INT (FK)
created_at	TIMESTAMP

SCHEMA NOTE Every table carries created_at — enables ranking, trending, and chronological queries from day one.

Relationship Explanation & Data Flow

How the entities connect, and what happens when a user takes action.

Relationship Summary

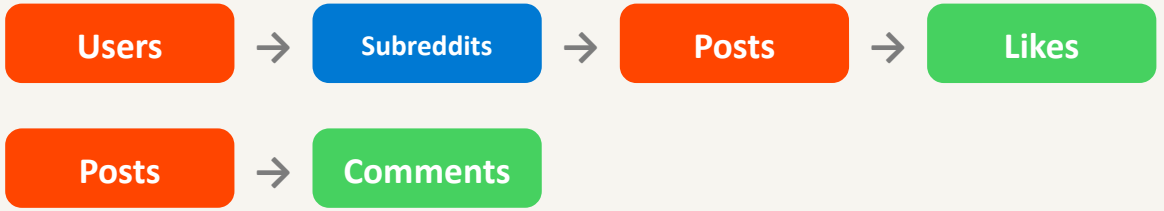
FROM	TO	TYPE
User	Subreddit	1 : M
User	Post	1 : M
User	Comment	1 : M
User	Like	1 : M
Subreddit	Post	1 : M
Post	Comment	1 : M
Post	Like	1 : M

SCENARIO

User "Alex" walks through the platform

1. Creates a subreddit called "Technology"
2. Publishes a post inside it
3. Another user likes the post
4. Multiple users add comments

Database Impact



Each action writes to one or more tables — chained inserts keep the graph consistent.

DATA FLOW INSIGHT A single user journey touches up to four tables — relationship integrity is non-negotiable at scale.

Question 4: Business Questions & SQL Mapping

Understand the business need first — then translate it into SQL.

Business Requirement → SQL Operation

BUSINESS REQUIREMENT	SQL OPERATION
Show recent posts in a subreddit	SELECT + JOIN + ORDER BY
Find most-liked post	COUNT + GROUP BY
Find subreddits created by a user	COUNT + WHERE
Find user with most likes given	COUNT + GROUP BY
Find posts containing URLs	LIKE operator

PM PERSPECTIVE

Why SQL Matters for PMs

- **Self-service**
  Run your own data pulls without waiting on the team.
- **Less dependency**
  Reduce backlog on analytics & engineering.
- **Validate fast**
  Test product hypotheses with real numbers.
- **Drive decisions**
  Move from gut-feel to evidence-based bets.

BRIDGE Good PMs translate user needs into entities, entities into queries, queries into shipped insight.

SQL Solutions — Content & Engagement Analysis

Three queries answering: what's new, what wins, what users build.

Q1 Recent 10 posts in a subreddit

```sql
SELECT
  p.post_id,
  p.title,
  p.created_at,
  u.username
FROM Posts p
JOIN Users u
  ON p.user_id = u.user_id
WHERE p.subreddit_id = 101
ORDER BY p.created_at DESC
LIMIT 10;
```

Q2 Most-liked post ever

```sql
SELECT p.post_id, p.title,
  COUNT(l.like_id) AS total_likes
FROM Posts p
JOIN Likes l ON p.post_id = l.post_id
GROUP BY p.post_id, p.title
ORDER BY total_likes DESC LIMIT 1;
```

Q3 Subreddits created by a user

```sql
SELECT COUNT(*) AS total_subreddits
FROM Subreddits
WHERE created_by = 1;
```

QUERY PATTERN JOIN + GROUP BY + ORDER unlocks ranking — the same shape powers trending, top, and leaderboards

SQL Solutions — User Activity Analysis

Who's most active, and which posts carry external links.

Q4 User who has given the most likes

```sql
SELECT
  u.user_id,
  u.username,
  COUNT(l.like_id) AS total_likes_given
FROM Users u
JOIN Likes l ON u.user_id = l.user_id
GROUP BY u.user_id, u.username
ORDER BY total_likes_given DESC LIMIT 1;
```

Q5 Posts containing URLs

```sql
SELECT post_id, title, description
FROM Posts
WHERE description LIKE '%http%';
```

PM INSIGHT

These queries answer:

1. **Which content is trending?**
   Q1 + Q2 power ranking & feed.
2. **Who are the power users?**
   Q4 identifies top likers / contributors.
3. **Which communities are growing?**
   Q3 surfaces user-driven community creation.
4. **What content drives engagement?**
   Q2 + Q5 separate winners and link-rich posts.

BUSINESS VALUE Five queries — content discovery, ranking, growth signals, and engagement diagnostics, end-to-end.

Question - 5

Reddit × DuckDuckGo Search Partnership

Surface Reddit discussions inside DuckDuckGo search — Reddit shares the ad revenue from the resulting traffic.

PLATFORM

For Reddit

- Increase traffic from external search engines
- Improve content discoverability
- Grow user acquisition

SEARCH

For DuckDuckGo

- Improve search-result quality
- Surface real community discussions
- Increase user engagement

AUDIENCE

For Users

- Discover authentic discussions
- Access community-generated answers
- Find relevant content faster

Value Loop



PARTNERSHIP THESIS Three winners, one loop: Reddit gains traffic, DuckDuckGo gains quality, users get authentic answers.

Search Integration API Design

Single REST endpoint that DuckDuckGo calls to fetch ranked Reddit posts.

GET `/api/v1/search/posts`

REQUEST PARAMETERS

PARAMETER	TYPE	DESCRIPTION
query	string	Search keyword
limit	integer	Number of results
sort	string	relevance / top / recent
subreddit	string	Optional subreddit filter

EXAMPLE REQUEST

```
GET /api/v1/search/posts?query=best+laptop&limit=5
```

AUTHENTICATION

API

API Key Authentication

Only authorized DuckDuckGo systems can call the endpoint.

- Per-partner keys
- Rate-limited
- Rotated on schedule

API CONTRACT One endpoint, four parameters, key-based auth — small surface area, easy to version and monitor.

Sample API Response

Just enough data to render a result — no more, no less.

response.json

```
{
  "query": "best laptop",
  "results": [
    {
      "post_id": 101,
      "title": "Best Laptop for Students",
      "subreddit": "technology",
      "author": "john_doe",
      "upvotes": 1250,
      "url": "reddit.com/post/101"
    }
  ]
}
```

RETURNED INFORMATION

- Post Title
- Subreddit
- Author
- Upvotes
- Reddit URL
- Ranking Score

WHY THIS DESIGN?

Just enough data for DuckDuckGo to render a result card — no markdown, no comments, no media payloads.

→ *Lower latency · smaller payloads · cheaper egress*

RESPONSE PRINCIPLE Return what the consumer renders — don't ship internal fields, debug data, or full post bodies.

Search Request Flow

From the user's query to the click on a Reddit link — every hop in between.

1. **User searches "best laptop for students"**
   Query enters the funnel
2. **DuckDuckGo Search Engine**
   Routes query to partner APIs
3. **Reddit Search API**
   Authenticates + validates request
4. **Reddit Search Service**
   Ranks & filters candidates
5. **PostgreSQL + Elasticsearch**
   Stores entities + full-text index
6. **Relevant Reddit Posts**
   JSON response returned
7. **DuckDuckGo Search Results**
   Rendered alongside DDG output
8. **User Clicks Reddit Link**
   Referral param captured

Key Systems Involved

- **DuckDuckGo Search Engine**
  External partner
- **Reddit API Gateway**
  Auth + rate limiting
- **Search Service**
  Ranking + filtering
- **Elasticsearch**
  Full-text retrieval
- **PostgreSQL**
  Source of truth

Each box owned by a different team — clear interfaces, isolated failure domains.

FLOW INSIGHT Eight hops, sub-second budget — observability at every boundary is non-negotiable.

Revenue Sharing & Click Attribution

CHALLENGE

How will Reddit know which traffic came from DuckDuckGo?

SOLUTION

Add a referral parameter: `reddit.com/post/101?source=duckduckgo`

Tracking Flow

DuckDuckGo Search → User Clicks Result → Referral Captured → Traffic Logged → Revenue Attribution → Monthly Report

Metrics Tracked

METRIC	PURPOSE
Referral Clicks	Traffic attribution
Click-Through Rate (CTR)	Search-result effectiveness
Session Duration	Content quality
Ad Revenue Generated	Revenue sharing
Conversion to Reddit Signup	User acquisition

PM RECOMMENDATION

Build a dedicated Attribution Service

One service that measures referral traffic, engagement, and revenue accurately — exposed as an internal API.

- **Single source of truth**
- **Auditable revenue logs**

ATTRIBUTION PRINCIPLE Every click should be traceable to revenue — without it, partnerships can't be priced or renewed.

Key Recommendations

Four areas that determine whether Reddit scales gracefully — or painfully.

1

ARCHITECTURE

Technology Architecture

- Microservices for independent service scaling
- React for a responsive front-end
- PostgreSQL for transactional consistency
- Redis for caching hot content
- Elasticsearch for fast, relevant search

2

DATA

Database Design

- Clear relationships across the five core entities
- PKs and FKs to enforce integrity
- Schemas built for high-volume community traffic

3

PARTNERSHIPS

Search Partnership API

- Secure REST API for external search partners
- API key auth + per-partner rate limiting
- Referral tracking for accurate revenue attribution

4

SCALE

Scalability Considerations

- Cache popular subreddit content
- Horizontally scale backend during traffic spikes
- Optimize search indexing for trending discussions

SUMMARY Build for scalability, maintain strong data integrity, enable ecosystem growth through secure integrations.

DECISION FRAME Pick technology by product fit; build APIs as products; treat scale as a feature, not an afterthought.

Final Takeaways

END OF DECK

Product thinking · Technical depth · PM judgment — three lenses, one cohesive design.

What this exercise demonstrates — three lenses on the same problem.

01

LENS 1

Product Thinking

- Understanding Reddit's user ecosystem
- Translating business requirements into technical solutions
- Designing scalable product architecture

02

LENS 2

Technical Understanding

- Technology stack evaluation
- Database schema design
- SQL-based data analysis
- API design and integration planning

03

LENS 3

PM Decision-Making

- Evaluating trade-offs between technologies
- Balancing scalability, performance, and cost
- Aligning technical design with business objectives

END OF DECK Product thinking · Technical depth · PM judgment — three lenses, one cohesive design.